

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

レポート課題：PKI に基づくデータ暗号化基盤の構築とセキュリティ評価

(出題日：2025 年 12 月 19 日(金)，提出日：2026 年 1 月 7 日(水))

学籍番号：v25y0009

氏 名：Davaakhuu Undarmaa

● 課題の目的

本演習で実施した「Private CA の構築と Web サーバ証明書の発行」を通じて、PKI の構成要素、TLS 通信における暗号化の仕組み、証明書運用上の課題について理解・考察することを目的とする。

● 設問 1: PKI とデータ暗号化の関係

1.1 PKI の構成要素とその役割

PKI は、公開鍵暗号技術を基盤とした信頼のフレームワークであり、以下の要素から構成される。

(1) 認証局(CA: Certificate Authority)

認証局は、PKI における信頼の起点として機能する組織または実体である。その主要な役割は以下の通りである。

第一に、証明書発行申請者の身元確認を行う。商用 CA の場合、ドメイン所有権の確認 (DV: Domain Validation)、組織の実在性確認 (OV: Organization Validation)、厳格な本人確認 (EV: Extended Validation) など、複数の検証レベルが存在する。Private CA の場合は、組織内のポリシーに基づいた検証が行われる。

第二に、デジタル証明書の発行と署名を実施する。CA は自身の秘密鍵を用いて証明書に署名することで、「この証明書に記載された公開鍵は、記載された主体のものである」という保証を提供する。

第三に、証明書のライフサイクル管理を担う。具体的には、証明書の発行、更新、失効処理 (CRL: Certificate Revocation List や OCSP: Online Certificate Status Protocol による)、アーカイブなどが含まれる。

(2) サーバ証明書

サーバ証明書は、Web サーバなどのサービス提供者の身元と公開鍵を証明するデジタル文書である。X.509 標準に準拠した形式で発行され、以下の情報を含む。

主体情報として、Common Name (CN: 通常は FQDN)、Organization (O)、Organizational Unit (OU)、Country (C) などが記載される。公開鍵情報として、公開鍵そのものとアルゴリズム (RSA、ECDSA など) およびキー長が含まれる。発行者情報として、証明書に署名した CA の識別名が記録される。有効期間として、証明書の有効開始日時と終了日時が定義される。署名情報として、CA の秘密鍵による署名値とアルゴリズムが付与される。

セキュリティ実践演習第一(2025 年度後期集中) 「データの暗号化」レポート課題

サーバ証明書の本質的な役割は、クライアントがサーバの正当性を検証できるようにすることである。これにより、中間者攻撃やなりすましを防止できる。

(3) 公開鍵・秘密鍵

公開鍵暗号方式における鍵ペアは、PKI の暗号学的基盤を構成する。

秘密鍵は、所有者のみが保持する機密情報であり、以下の用途に使用される。デジタル署名の生成により、文書やメッセージの作成者を証明し、改ざん検知を可能にする。公開鍵で暗号化されたデータの復号により、送信者が意図した受信者のみがデータを読めることを保証する。

TLS handshake においては、鍵交換プロセスに参加する。

公開鍵は、広く配布される情報であり、証明書に含まれる。その用途として、秘密鍵で生成された署名の検証、データの暗号化（受信者の公開鍵で暗号化することで、その秘密鍵保持者のみが復号可能）、TLS handshake における鍵交換への参加がある。

公開鍵と秘密鍵は数学的に関連しているが、一方から他方を現実的な時間で計算することは困難である（計算量的安全性）。この非対称性がセキュリティの根幹となっている。

(4) ルート証明書

ルート証明書は、認証局階層構造の最上位に位置する自己署名証明書である。自己署名とは、発行者と主体が同一であり、自身の秘密鍵で自身の証明書に署名している状態を指す。

ルート証明書の特徴として、信頼の起点（Trust Anchor）として機能する点が挙げられる。クライアント（OS やブラウザ）は、あらかじめ信頼されたルート証明書のリストを保持しており、この「信頼されたルート証明書ストア」に含まれる CA のみを信頼する。

商用ルート CA の証明書は、OS・ブラウザベンダーによる厳格な審査を経てプレインストールされる。Private CA の場合は、管理者が手動で信頼ストアに追加する必要がある。

ルート秘密鍵は極めて重要な資産であり、通常はオフライン環境で厳重に保管され、HSM（Hardware Security Module）などの物理的セキュリティデバイスに格納される。

(5) 証明書チェーン

証明書チェーンは、エンドエンティティ証明書（サーバ証明書など）からルート証明書までの信頼の連鎖を表す構造である。

典型的な階層構造は以下の通りである。ルート証明書（自己署名、最上位）が中間証明書（ルート CA により署名、実際の証明書発行業務を担当）に署名し、中間証明書がエンドエンティティ証明書（中間 CA により署名、実際のサーバやクライアントで使用）に署名する。

証明書チェーンの検証プロセスは次のように進む。クライアントは、サーバから受信したサーバ証明書の署名を、発行者（中間 CA）の公開鍵を用いて検証する。次に、中間 CA 証明書の署名を、その発行者（ルート CA）の公開鍵を用いて検証する。最終的に、ルート証明書が信頼ストアに存在することを確認する。全てのステップで検証が成功し、証明書が有効期限内であれば、チェーン全体が有効と判断される。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

階層構造の利点として、ルート秘密鍵の使用頻度を最小化できる点、中間 CA を用途別に分割できる点（例：サーバ証明書用、コード署名用など）、中間 CA 証明書が侵害された場合でも、その中間 CA のみを失効させることで影響を局所化できる点が挙げられる。

1.2 TLS 通信における暗号化方式の使い分け

TLS (Transport Layer Security) 通信では、公開鍵暗号と共通鍵暗号が異なる目的で使い分けられる。この設計は、各暗号方式の特性を最適に活用するためである。

(1) 公開鍵暗号の役割

公開鍵暗号は、TLS handshake 段階で以下の目的に使用される。

第一に、サーバ認証である。サーバは自身の証明書をクライアントに送信し、クライアントは証明書チェーンを検証することでサーバの正当性を確認する。証明書に含まれる公開鍵が真にそのサーバのものであることが保証される。

第二に、鍵交換である。TLS 1.2 以前では、RSA 鍵交換方式や Diffie-Hellman (DH) 鍵交換が使用される。RSA 鍵交換では、クライアントが共通鍵の元となる Pre-Master Secret を生成し、サーバの公開鍵で暗号化してサーバに送信する。DHE (Ephemeral Diffie-Hellman) や ECDHE (Elliptic Curve DHE) では、サーバが鍵交換パラメータを秘密鍵で署名し、クライアントが公開鍵で検証した後、両者が鍵交換を行う。TLS 1.3 では、鍵交換は ECDHE または DHE のみに限定され、forward secrecy (前方秘匿性) が必須となっている。

第三に、オプションのクライアント認証である。クライアント証明書認証を使用する場合、クライアントも証明書を提示し、秘密鍵で署名を行うことで自身を証明する。

公開鍵暗号の特性として、鍵配送問題を解決できる点、認証と鍵交換を同時に実現できる点が挙げられる一方、計算コストが高く処理速度が遅いという欠点がある。RSA 2048 ビットの暗号化・復号は、AES-128 の数百倍から数千倍の計算量を要する。

(2) 共通鍵暗号の役割

共通鍵暗号は、TLS handshake 完了後の実際のデータ通信段階で使用される。

handshake で安全に共有された鍵情報から、実際の暗号化に使用するセッション鍵（共通鍵）が導出される。具体的には、Pre-Master Secret (TLS 1.2) または shared secret (TLS 1.3) から、Master Secret が生成され、さらにこれから複数のセッション鍵（暗号化鍵、MAC 鍵など）が導出される。

このセッション鍵を用いて、HTTP 通信の内容が暗号化される。使用されるアルゴリズムは、TLS 1.2 では AES-CBC、AES-GCM、ChaCha20-Poly1305 などである。TLS 1.3 では、AEAD (Authenticated Encryption with Associated Data) 方式のみが許可され、AES-GCM や ChaCha20-Poly1305 が使用される。共通鍵暗号の特性として、処理速度が非常に高速である点が挙げられる。ハードウェアアクセラレーションにより、さらに高速化が可能である。また、大量データの暗号化に適しているという利点がある。一方、鍵配送問題が存在するため、安全な鍵共有方法が必要となる欠点がある。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

(3) hybrid 暗号方式の利点

TLS が公開鍵暗号と共通鍵暗号を組み合わせる理由は、以下の通りである。
セキュリティ面では、公開鍵暗号により、事前の秘密共有なしに安全な通信路を確立できる。
共通鍵の安全な配送が実現され、中間者攻撃に対する防御が可能となる。パフォーマンス面では、公開鍵暗号の使用を鍵交換と認証のみに限定することで、オーバーヘッドを最小化できる。
大量データの暗号化には高速な共通鍵暗号を使用することで、実用的な通信速度を実現できる。
前方秘匿性の実現も重要である。セッション毎に異なる一時的な鍵を使用することで、長期秘密鍵が漏洩しても過去の通信内容は保護される。この設計により、TLS は「セキュリティ」「パフォーマンス」「スケーラビリティ」のバランスを実現している。

● 設問 2: 演習で構築したプライベート CA 構築手順

(2.1) 演習で構築した Private CA の構成を図示し、処理の流れを説明。

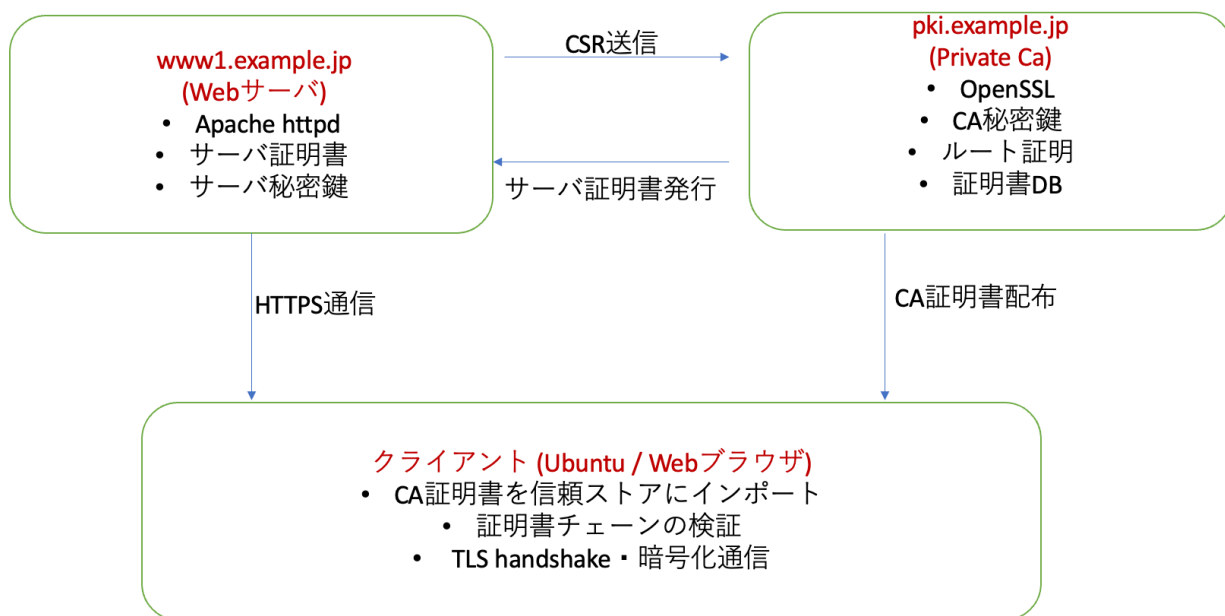


図 1: Private CA 構成の図示・流れに示す

処理フロー

[準備段階]

1. CA サーバ: CA 秘密鍵の生成
2. CA サーバ: ルート証明書の作成
3. Web サーバ: サーバ秘密鍵の生成

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

[証明書発行段階]

4. Web サーバ → CA サーバ: CSR 送信
5. CA サーバ: 本人確認・審査
6. CA サーバ: CSR に署名してサーバ証明書発行
7. CA サーバ → Web サーバ: サーバ証明書送付

[配備段階]

8. Web サーバ: Apache 設定ファイルに証明書を指定
9. Web サーバ: httpd サービス再起動
10. クライアント: CA 証明書(ルート証明書)を信頼ストアにインポート

[運用段階]

11. クライアント → Web サーバ: HTTPS 接続 (https://www1.example.jp/)
12. TLS handshake: 証明書チェーン検証
13. 暗号化通信の確立。

example.jp ドメイン配下に, DNS サーバ (localdns.example.jp) , Private CA サーバ (pki.example.jp) , Web サーバ (www1.example.jp) を配置し, すべて同一ネットワークセグメント (10.0.0.0/24) 上で構成した。クライアントとして Ubuntu ノート PC を用い, 無線 LAN (SecurityLab) を介して演習用ネットワークに接続した。

1. CA 用秘密鍵の生成

本演習では, Private CA を構築するために, easy-rsa を用いて CA 用の秘密鍵および自己署名証明書を作成した。以下に, 実際に実行したコマンド例とその目的を示す。

1.1 easy-rsa のインストール

```
sudo apt install easy-rsa
```

easy-rsa は, OpenSSL を内部で利用し, CA 構築や証明書管理を簡易化するツールである。本演習では, Private CA の構築を容易にするために使用した。

1.2 PKI 用ディレクトリの作成

```
mkdir ~/easy-rsa
```

CA 関連の鍵や証明書を管理するための専用ディレクトリを, security ユーザのホームディレクトリ配下に作成した。

1.3 symbolic link の作成

```
ln -s /usr/share/easy-rsa/* ~/easy-rsa/
```

システムにインストールされた easy-rsa の実体ファイルを, 作業ディレクトリから利用できるようにするため, シンボリックリンクを作成した。

1.4 ディレクトリ権限の設定

```
chmod 700 ~/easy-rsa
```

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

CA の秘密鍵を安全に管理するため、ディレクトリのアクセス権を所有者のみが操作可能な 700 に設定した。

1.5 PKI ディレクトリの初期化

```
cd ~/easy-rsa
./easysrsa init-pki
```

証明書および鍵を格納するための PKI ディレクトリ構造を初期化した。この操作により、pki ディレクトリが生成される。

1.6 vars ファイルの編集

```
vi vars
```

CA および発行する証明書に記載される組織情報を設定した。具体的には、国名、組織名、部局名などに加え、暗号アルゴリズムとして楕円曲線暗号 (EC) , ハッシュ関数として SHA-512 を指定した。これにより、証明書の識別性および暗号学的安全性を高めている。

```
countryName          :PRINTABLE:'JP'
stateOrProvinceName  :ASN.1 12:'Oita'
localityName         :ASN.1 12:'Oita'
organizationName     :ASN.1 12:'Oita University'
organizationalUnitName:ASN.1 12:'Community'
commonName           :ASN.1 12:'www1.example.jp'
emailAddress         :IA5STRING:'v25y0009@oita-u.ac.jp'
```

1.7 CA の構築

```
./easysrsa build-ca
```

CA 用の秘密鍵と自己署名証明書を生成した。

CA 用の秘密鍵 (ca.key) と自己署名証明書 (ca.crt) を生成した。この際、CA 秘密鍵にはパスフレーズを設定し、安全な場所に記録した。Common Name (CN) はデフォルト値を使用した。

1.8 CA 証明書および秘密鍵の確認

```
cd pki/ls
```

ca.crt (CA 公開証明書) および ca.key (CA 秘密鍵) が生成されていることを確認した。

1.9 CA 公開証明書の配布

```
cat ~/easy-rsa/pki/ca.crt
```


セキュリティ実践演習第一(2025 年度後期集中) 「データの暗号化」レポート課題

表示された「BEGIN CERTIFICATE」から「END CERTIFICATE」までの内容をすべてコピーし、Web サーバ (www1.example.jp) へ転送した。これにより、Web サーバ側で Private CA によって発行された証明書を信頼できるようになり、HTTPS 通信の基盤が整備された。

1.10 Private CA 公開証明書の登録

本手順では、Private CA で作成した CA 公開証明書 (ca.crt) を、Web サーバ (www1.example.jp) 側に登録し、Private CA を信頼できる認証局として認識させる設定

```
sudo cp /tmp/ca.crt /etc/pki/ca-trust/source/anchors/  
sudo update-ca-trust
```

コマンドは、Private CA の公開証明書 (ca.crt) を、RHEL 系 Linux における信頼された認証局証明書の格納ディレクトリである。このディレクトリに配置された証明書は、「信頼された CA」として扱われる対象となる。コマンドは、システムに登録されている CA 証明書の情報を更新し、先ほど配置した Private CA の公開証明書をシステム全体の信頼ストアに反映させるものである。

これにより、OS および Apache httpd などのアプリケーションが、Private CA によって発行されたサーバ証明書を正当なものとして検証できるようになる。

これらの操作により、Web サーバ側において Private CA が正式に信頼される状態となり、Private CA が発行したサーバ証明書を用いた HTTPS 通信が可能となる。

2. サーバ証明書署名要求 (CSR) の作成

本演習では、Web サーバ (www1.example.jp) 上でサーバ証明書署名要求 (CSR: Certificate Signing Request) を作成した。CSR は、サーバの公開鍵および識別情報 (組織名、Common Name 等) を含み、CA に対して「このサーバ用の証明書を発行してほしい」という要求を行うためのデータである。

(2) サーバ秘密鍵の生成

以下のコマンドを用いて、サーバ証明書用の秘密鍵を生成した。

```
openssl genrsa -out www1.key 2048
```

このコマンドでは、RSA アルゴリズムを用いて 2048bit 長の秘密鍵を生成している。2048bit RSA は、現在一般的に利用されている安全性と性能のバランスが取れた鍵長である。

本研究では、CA (認証局) の構築には easy-rsa を使い、楕円曲線暗号 (EC) および SHA-512 ハッシュアルゴリズムを使用した。一方、Web サーバ証明書用の秘密鍵については、OpenSSL を用いて RSA 2048bit 鍵を生成した。

生成された www1.key はサーバの秘密鍵であり、Web サーバ (www1.example.jp) 上にのみ保存し、CA サーバ (pki.example.jp) へは決して転送しない。これは、秘密鍵が漏えいした場合、第三者によるなりすまし通信が可能になるためである。

(2.2) CSR (証明書署名要求) の作成

次に、生成した秘密鍵を用いて CSR を作成した。

```
openssl req -new -key www1.key -out www1.csr
```

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

この操作により、サーバの公開鍵および証明書に記載される情報を含む CSR ファイル `www1.csr` が生成される。CSR 作成時に入力する項目のうち、Common Name (CN) には「`www1.example.jp`」を指定した。

```
[security@www1 ~]$ ls
www1.csr  www1.key
```

(2.3) CSR の CA サーバへの転送

作成した CSR を、Private CA が稼働している `pki.example.jp` へ転送した。

`scp pki/issued/www1.crt security@www1.example.jp:/tmp/`

```
security@pki:~/easy-rsa$ scp pki/issued/www1.crt security@www1.example.jp:/tmp/
The authenticity of host 'www1.example.jp (10.0.0.153)' can't be established.
ED25519 key fingerprint is SHA256:c7zJXceKaHoyCmPec7FohdpnGi3QGETMM20gYCvsV5I.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'www1.example.jp' (ED25519) to the list of known hosts.
security@www1.example.jp's password:
www1.crt
```

この操作により、Private CA で署名されたサーバ証明書 (`www1.crt`) を、Web サーバ側で利用可能にした。

(3) CA によるサーバ証明書の署名

```
cd ~/easy-rsa ./easyrsa import-req /tmp/www1.csr www1
./easyrsa sign-req server www1
```

Web サーバから `scp` で転送された証明書署名要求ファイル (`www1.csr`) を `easy-rsa` の `import-req` コマンドを用いて CA 環境に取り込んだ。その後、`sign-req server` コマンドを実行し、CA の秘密鍵 (`ca.key`) を用いて CSR に対する署名処理を行った。サーバ証明書 (`www1.crt`) が生成され、Web サーバで HTTPS 通信を行うための証明書が発行された。

```
Type the word 'yes' to continue, or any other input to abort.
Confirm request details: yes

Using configuration from /home/security/easy-rsa/pki/openssl-easyrsa.cnf
Enter pass phrase for /home/security/easy-rsa/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
```

図 2: `easy-rsa` を用いた CA による署名処理の実行結果

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

CA 秘密鍵 (ca. key) に設定したパスフレーズを入力し、証明書署名要求に対する署名処理が正常に完了したことを示している。「Signature ok」と表示されており、CA の秘密鍵を用いた電子署名が正しく行われたことが確認できる。

4. Web サーバ(Apache httpd)への証明書設定

目的:Private CA によって発行したサーバ証明書を Apache httpd に設定し、HTTPS (TLS) による暗号化通信を実現することを目的とする。

まず、www1.example.jp 上で Apache HTTP Server (httpd) の基本設定を行い、サービスを起動した。

(1)httpd サービスの起動確認

```
sudo systemctl start httpd
```

次に、Web ブラウザから以下の URL へアクセスし、ページが表示されることを確認した。

http://www1.example.jp/

表示できない場合には、以下の点を確認した。

```
Complete!
[security@www1 ~]$ sudo systemctl start httpd
[security@www1 ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Wed 2025-12-24 13:36:05 JST; 38s ago
     Docs: man:httpd.service(8)
  Main PID: 19691 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:  0 B/sec"
    Tasks: 177 (limit: 10652)
  Memory: 16.1M (peak: 16.3M)
    CPU: 46ms
```

httpd サービスが起動しているか

firewall の設定で HTTP (80/tcp) が許可されているか

(2) firewall の設定 (HTTP / HTTPS 通信の許可)

```
sudo firewall-cmd --permanent --add-service=http/ https
```

Web サーバが外部からの HTTP / HTTPS アクセスを受け付けられるようにするため、firewall において必要なポートを許可

```
[security@www1 ~]$ sudo firewall-cmd --permanent --add-service=http
success
[security@www1 ~]$ sudo firewall-cmd --reload
success
[security@www1 ~]$ sudo firewall-cmd --list-services
cockpit dhcpv6-client http ssh
```

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

HTTP 通信 (TCP 80 番ポート) を firewall で恒久的に許可した。
初期動作確認やリダイレクト用途に使用. TLS による暗号化通信
(TCP 443 番ポート) を許可し, HTTPS 接続を可能にした.

(3) html へのファイル配置と動作確認

Apache のドキュメントルートである /var/www/html 以下に, 自身で作成した index.html を配置した。その後, Web ブラウザからアクセスし, 作成した HTML コンテンツが正しく表示されることを確認した。

`sudo cp index.html /var/www/html/`

```
[security@www1 ~]$ sudo cp /tmp/index.html /var/www/html/  
[sudo] password for security:  
[security@www1 ~]$ ls -l /var/www/html/index.html  
-rw-r--r-- 1 root root 142 Dec 24 13:50 /var/www/html/index.html
```

Apache が公開するディレクトリ (DocumentRoot) に Web コンテンツを配置するためにコピーを行った。

```
<html>  
<head><title>My Web Page</title></head>  
<body>  
<h1>Hello from www1.example.jp</h1>  
<p>This page was created by me.</p>  
</body>  
</html>
```

Hello from www1.example.jp

This page was created by me.

/var/www/html に index.html を配置し, Web サーバが正常にコンテンツを配信できることを確認した。

実行したコマンド (www1.example.jp 上)

- `sudo cp /tmp/www1.crt /etc/pki/tls/www1/`
- `sudo cp www1.key /etc/pki/tls/www1/`
- `sudo vi /etc/httpd/conf.d/ssl.conf`
- `sudo systemctl restart httpd`

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

```
[security@www1 ~]$ scp www1.csr security@pki.example.jp:/tmp/
security@pki.example.jp's password:
Permission denied, please try again.
security@pki.example.jp's password:
www1.csr
[security@www1 ~]$ sudo mkdir -p /etc/pki/tls/www1
[sudo] password for security:
[security@www1 ~]$ sudo cp /tmp/www1.crt /etc/pki/tls/www1/
[security@www1 ~]$ sudo cp www1.key /etc/pki/tls/www1/
[security@www1 ~]$
```

Apache 設定ファイル (ssl.conf)

```
<VirtualHost _default_:443>

# General setup for the virtual host, inherited from global configuration
#DocumentRoot "/var/www/html"
#ServerName www1.example.jp:443

# Use separate log files for the SSL virtual host; note that LogLevel
# is not inherited from httpd.conf.
ErrorLog logs/ssl_error_log
TransferLog logs/ssl_access_log
LogLevel warn

#    SSL Engine Switch:
#    Enable/Disable SSL for this virtual host.
SSLEngine on
SSLCertificateFile /etc/pki/tls/www1/www1.crt
SSLCertificateKeyFile /etc/pki/tls/www1/www1.key

#    List the protocol versions which clients are allowed to connect with.
```

Private CA によって署名されたサーバ証明書 (www1.crt) および, Web サーバ上で生成した秘密鍵 (www1.key) を, Apache httpd が参照可能なディレクトリに配置した。

次に, Apache の SSL 設定ファイル (ssl.conf) において SSLEngine on を設定し, SSL/TLS 通信を有効化した。

さらに, 以下の証明書関連ファイルを指定した。

- SSLCertificateFile : サーバ証明書 (www1.crt)
→ クライアントへ送信され, サーバの正当性を示すために使用される。
- SSLCertificateKeyFile : サーバ秘密鍵 (www1.key)

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

→ TLS ハンドシェイクにおける鍵交換および電子署名に使用される。以上の設定により、Apache httpd 上で Private CA に基づく HTTPS 通信を実現した。

```
[security@www1 ~]$ sudo vi /etc/httpd/conf.d/ssl.conf
[security@www1 ~]$ sudo apachectl configtest
Syntax OK
[security@www1 ~]$
```

ブラウザから www1.example.jp にアクセスしたところ、Apache の Test Page が表示され、Web サーバが正常に動作していることを確認した。

- 設問 3: サーバ証明書による暗号化通信の確認

- 3.1 HTTPS 接続時に確認したサーバ証明書情報

本演習では、自身が構築した Web サーバに対して HTTPS 接続を行い、openssl および証明書生成時のログから、サーバ証明書の情報を確認した。

- (1) 証明書情報

```
Using configuration from /home/security/easy-rsa/pki/openssl-easyrsa.cnf
Enter pass phrase for /home/security/easy-rsa/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'JP'
stateOrProvinceName   :ASN.1 12:'Oita'
localityName          :ASN.1 12:'Oita'
organizationName       :ASN.1 12:'Oita University'
organizationalUnitName:ASN.1 12:'Community'
commonName            :ASN.1 12:'www1.example.jp'
emailAddress          :IA5STRING:'v25y0009@oita-u.ac.jp'
Certificate is to be certified until Mar 28 04:27:21 2028 GMT (825 days)

Write out database with 1 new entries
```

- (1) 証明書の発行者 (Issuer)

本 Web サーバで使用したサーバ証明書は、easy-rsa を用いて構築した Private CA により発行された。

Issuer: CN=Easy-RSA CA, O=Oita University, C=JP

- Using configuration from ... ca.key_ Enter pass phrase for ... ca.key

このことは、サーバ証明書作成時に CA の秘密鍵 (ca.key) を用いて署名が行われているログから確認できる。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

(2) 有効期限 (Validity)

サーバ証明書の有効期限は以下の通りである。

[Certificate is to be certified until Mar 28 04:27:21 2028 GMT (825 days)] つまり、本証明書は発行日から約 825 日間有効である。

(3) 使用されている暗号アルゴリズム

アルゴリズム	鍵長	安全性レベル (ビット)	備考
RSA	2048 ビット	112 ビット	現在の標準。2030 年頃まで安全とされている
RSA	3072 ビット	128 ビット	より長期的な安全性を確保可能
RSA	4096 ビット	152 ビット	ルート CA や中間 CA での利用に推奨
ECDSA P-256	256 ビット	128 ビット	高性能・低負荷。モバイル端末向け
ECDSA P-384	384 ビット	192 ビット	非常に高いセキュリティ要件向け
Ed25519	256 ビット	128 ビット	最新方式。高速かつ安全性が高い

表 1: 暗号アルゴリズム別 鍵長・安全性レベル比較表

演習では互換性と実装の容易さを考慮し、Ed25519 ビットを使用した。

アルゴリズム	安全性	備考
SHA-1	脆弱	2017 年に衝突攻撃が実証されており、現在は使用禁止
SHA-256	安全	現在の標準。広い互換性と十分な安全性を持つ
SHA-384	安全	より高いセキュリティ要件向け
SHA-512	安全	安全だが、処理負荷が高くオーバースペック気味

表 2: ハッシュ関数 (署名用) の安全性比較表

本演習では互換性と実運用を考慮し、署名用ハッシュ関数として SHA-256 を使用した。

本演習では、easy-rsa の設定ファイルにおいて以下の暗号設定を用いた。

```
The authenticity of host 'pki.example.jp (10.0.0.152)' can't be established.  
ED25519 key fingerprint is SHA256:DArhVM+EMHXu5cfIgb+8LbXoTEWx2l9HLB682jdGamU.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'pki.example.jp' (ED25519) to the list of known hosts.  
security@pki.example.jp's password:  
stat local "www1.scr": No such file or directory  
[security@www1 ~]$ scp www1.scr security@pki.example.jp:/tmp/  
security@pki.example.jp's password:
```

署名アルゴリズム: ED25519

ハッシュ: SHA-256

なお、SSH 接続時に表示される SHA-256 はホスト公開鍵 fingerprint 用のハッシュ値であり、CA における証明書署名アルゴリズム (SHA-512) とは異なる用途で使用されている。

セキュリティ実践演習第一(2025 年度後期集中) 「データの暗号化」レポート課題

本演習では easy-rsa の設定において、署名アルゴリズムとして ED25519 が使用されており、ハッシュ関数には SHA-256 が用いられている。まず、SHA-256 により証明書の内容をハッシュ化し、そのハッシュ値に対して ED25519 による電子署名が行われる。これにより、証明書の改ざん検知および発行元の正当性が保証される。

(3.2) Web ブラウザでアクセスした際に、「警告」が表示された理由を、PKI の信頼モデルの観点から説明

「この接続ではプライバシーが保護されません」

NET::ERR_CERT_AUTHORITY_INVALID

URL: <https://www1.example.jp>

(Private CA により発行された証明書が信頼されていないため警告が表示された)

Web ブラウザから HTTPS 接続を行った際に、「この接続ではプライバシーが保護されません」という警告画面が表示された。これは、本演習で使用したサーバ証明書が Public CA ではなく、easy-rsa により構築した Private CA によって発行されたためである。PKI (Public Key Infrastructure) の信頼モデルでは、Web ブラウザは OS やブラウザにあらかじめ登録されている信頼済みのルート証明書を起点として証明書の正当性を検証する。しかし、本演習の Private CA のルート証明書はブラウザの信頼ストアに登録されていないため、証明書チェーンの検証に失敗し、ERR_CERT_AUTHORITY_INVALID が発生した。その結果、通信自体は暗号化されているものの、証明書の信頼性が保証されないため、警告画面が表示された。

```
sudo cp ca.crt /usr/local/share/ca-certificates/
```

```
sudo update-ca-certificates
```

本警告を解消するためには、Private CA のルート証明書 (ca.crt) をクライアント端末の信頼ストアに登録する必要があった。実際に ca.crt を OS または Web ブラウザへインポートすることで、証明書チェーンの検証が可能となり ERR_CERT_AUTHORITY_INVALID は解消される。



セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

● 設問 4: Private CA 運用におけるセキュリティ上の課

(1) セキュリティの観点

(Private CA の利点)

内部統制の強化: 証明書発行プロセスを組織が完全に管理できるため、外部 CA に依存せず独自のセキュリティポリシーを徹底できる
情報漏洩リスクの低減: 内部サーバ名やネットワーク構成などの機密情報を外部 CA (商用 CA) に提出する必要がない
カスタマイズ可能な認証: 組織特有の認証要件 (例: 特定の部署のみに発行、特殊な拡張フィールド) を自由に実装できる。
即時失効対応: 証明書侵害時に外部 CA への連絡を待たず、即座に失効処理を実施できる。

(Private CA の欠点)

管理者の技術力依存: CA 運用の安全性が管理者のスキルに大きく依存し、設定ミスや運用ミスがセキュリティホールとなる
第三者監査の欠如: 商用 CA のような WebTrust 監査などの第三者による厳格な監査がない
め、運用の適切性が 保証されないセキュリティ投資の負担: HSM 導入、物理的セキュリティ強化、監視システム構築など、すべて自己負担となる。CA 秘密鍵の保護責任: 組織が全責任を負うため、漏洩時の影響は甚大である。商用 CA のような保険や補償制度もない

2) 運用コストの観点

(Private CA の利点)

Private CA を導入することで、初期構築後は証明書を無制限に発行でき、証明書 1 枚あたりの発行コストが発生しない。そのため、多数の内部サーバを運用する環境では、大幅なコスト削減が可能となる。また、商用 CA における有効期間の制約 (現在は最大 398 日) を受けず、組織の運用方針に応じて柔軟に有効期間を設定できる点も利点である。
さらに、外部 CA への申請や審査プロセスが不要なため、証明書を短時間で発行でき、開発環境やテスト環境においても追加コストなしで繰り返し利用できる。

(Private CA の欠点)

一方で、Private CA の導入には初期構築コストが必要となる。
具体的には、CA サーバの構築、OpenSSL や easy-rsa の設定、ネットワーク設計などに時間と労力を要する。また、CA を安全に運用するためには、管理者の人件費や教育・訓練コストが継続的に発生する。加えて、サーバハードウェアや保守費用、バックアップ用ストレージ、監視ツールや証明書管理ツールの導入が必要となる場合がある。特に HSM (Hardware Security Module) を導入する場合には、高額なコストが発生する点も課題である。さらに、

証明書の有効期限管理、失効管理、ログ監査などの運用作業が継続的に発生し、運用負荷が増加する。

(3) 信頼性の観点

(Private CA の利点)

Private CA は組織内部で管理されるため、学内システムや社内システムなど、信頼関係が明確な環境においては十分な信頼性を確保できる。内部通信に限定した利用であれば、商用 CA を利用せずとも適切な認証基盤を構築可能である。また、冗長化構成やバックアップ戦略を組織の要件に応じて自由に設計できる点も利点である。さらに、商用 CA のサービス停止や企業買収、方針変更などの外部要因に依存せず、安定した運用が可能となる。

(Private CA の欠点)

一方で、Private CA の公開証明書は主要な Web ブラウザや OS の信頼ストアに事前登録されていないため、クライアント側で CA 証明書をインポートしない限り、証明書警告が表示される。そのため、全クライアント端末に対して CA 証明書を手動または自動配布する必要があり、端末数が多い環境では管理負荷が大きくなる。特に数百～数千台規模の環境では、GPO (Group Policy) や MDM ツールなどを用いた一括配布が不可欠となる。また、組織外の利用者（顧客や取引先など）に対しては、証明書エラーが発生するため、Private CA は適さない。さらに、商用 CA のような SLA（可用性保証）が存在せず、障害発生時の対応はすべて自己責任となる。加えて、複数拠点や複雑な組織構造を持つ場合、証明書チェーンの管理や検証が複雑化し、信頼関係の維持が困難になる可能性がある。

(4) 利便性の観点

(Private CA の利点)

Private CA では、外部 CA による審査や承認を待つ必要がなく、証明書を数分から数十分程度で迅速に発行できる。そのため、障害対応や緊急時にも即時対応が可能である。また、DN (Distinguished Name) や SAN (Subject Alternative Name) を組織の命名規則に合わせて柔軟に設定でき、有効期間についても業務要件に応じて短期・長期の調整が可能である。さらに、ACME プロトコルの導入やスクリプトによる運用自動化が容易であり、組織独自の API 連携や CI/CD 環境との統合にも適している。開発・テスト環境においては、証明書を何度でも再発行でき、実験的な設定を試行できる点も大きな利点である。

(Private CA の欠点)

一方で、Private CA の運用には初期学習コストが伴う。OpenSSL コマンドの操作に加え、PKI の理論、証明書形式、暗号アルゴリズムに関する知識が必要であり、管理者育成には時間を要する。また、新規端末の追加時には CA 証明書の配布作業が発生し、特にモバイル端末や IoT デバイスでは配布が困難な場合がある。さらに、OS ごとに手順が異なるため (Windows, macOS, Linux, iOS, Android), 運用負荷が増大する。証明書エラー発生時には、自組織で原因究明および対応を行う必要があり、商用 CA のような公式サポートが存在しない点も課題で

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

ある。加えて、古いシステムやレガシーアプリケーションでは、暗号アルゴリズムやプロトコルの非対応により互換性問題が生じる可能性がある。

(2) CA の秘密鍵が漏えいした場合に想定される影響と対策

1. 想定される影響

CA の秘密鍵が漏えいした場合、PKI 全体の信頼性が根本から崩壊する重大なインシデントとなる。具体的には、以下のような影響が想定される。まず、攻撃者が正規の CA を装い、不正なサーバ証明書やクライアント証明書を発行できるようになる。これにより、なりすましサーバの構築や、中間者攻撃 (Man-in-the-Middle attack) が可能となり、通信内容の盗聴や改ざんが発生する危険性が高まる。次に、既存の正当な証明書の信頼性も失われるため、すべての証明書を失効・再発行する必要がある。その結果、業務システムの停止や、大規模な運用負荷・コスト増加につながる。さらに、Private CA を利用している組織全体の信頼が損なわれ、内部システムだけでなく、外部との通信にも深刻な影響を及ぼす可能性がある。

2. 漏えいを防ぐための対策

CA 秘密鍵の漏えいを防ぐためには、技術的および運用的な複数の対策を組み合わせることが重要である。まず、CA 秘密鍵は厳格なアクセス制御のもとで管理し、権限を持つ管理者のみがアクセス可能とする。ファイル権限の最小化や、オフライン環境での CA 運用も有効な対策である。次に、秘密鍵を暗号化して保管し、強固なパスフレーズを設定することで、不正利用のリスクを低減できる。

さらに、HSM (Hardware Security Module) を導入することで、秘密鍵をハードウェア内に隔離し、外部への流出を防止することが可能である。また、CA サーバに対する定期的なログ監査や侵入検知システム (IDS/IPS) の導入により、不正アクセスの早期発見を行うことも重要である。加えて、万一漏えいが発生した場合に備え、証明書失効 (CRL/OCSP) および CA 再構築手順を事前に整備しておくことが、迅速な復旧につながる。

● 設問 5: (発展)実運用を想定した考察

● 公開 CA とプライベート CA の使い分け

公開 CA とプライベート CA の使い分けおよび証明書失効管理 (CRL/OCSP)

実運用環境においては、すべてのシステムに対して単一の CA を用いるのではなく、利用目的やアクセス主体に応じて 公開 CA とプライベート CA を適切に使い分ける設計 が重要である。まず、学外ユーザや不特定多数の利用者がアクセスする公開 Web サイトや EC サイトでは、主要な OS や Web ブラウザにあらかじめ信頼されている公開 CA を利用することが不可欠である。これにより、クライアント側での証明書警告を回避でき、高い利便性と信頼性を確保できる。一方で、学内システム、内部 API、VPN 接続、IoT デバイス間通信など、利用者および通信範囲が限定されている環境では、プライベート CA の利用が適している。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

- 証明書失効管理 (CRL/OCSP)

プライベート CA を用いることで、証明書発行の迅速化やコスト削減、および組織独自の命名規則や有効期間設定といった柔軟な運用が可能となる。このようなハイブリッド構成を採用する場合、証明書失効管理 CRL/OCSP が極めて重要となる。万一、サーバ証明書や秘密鍵が漏えいした場合、速やかに証明書を失効させなければ、中間者攻撃 (MITM) などの重大なセキュリティリスクが発生する。CRL は失効証明書の一覧を定期的に配布する方式であり、構成が単純で実装しやすい一方、証明書数が増加すると配布サイズが大きくなるという課題がある。一方、OCSP はリアルタイムで証明書の有効性を確認できるため、大規模環境や即時性が求められるシステムに適している。以上より、実運用においては外部向けサービスには公開 CA、内部システムにはプライベート CA を採用し、失効管理として CRL または OCSP を適切に組み合わせる設計が、セキュリティ、信頼性、運用効率の観点から有効であると考えられる。

学内システムへの Private CA 導入設計

(1) 証明書失効の必要性

証明書が侵害された場合、秘密鍵が漏洩した場合、証明書情報に誤りがあった場合、従業員が退職した場合などには、証明書を失効させる必要がある。失効管理が適切でないと、侵害された証明書が有効と判断され続け、セキュリティリスクが継続する。

(2) CRL (Certificate Revocation List) 方式

CRL は、CA が定期的に失効証明書のリストを公開する方式である。

仕組み: 証明書には、CRL 配布ポイント (CDP) の URL が含まれる。クライアントは定期的に CRL をダウンロードし、証明書のシリアル番号がリストに含まれているかを確認する。

設定例:

```
bash# CRL 生成
```

```
openssl ca -gencrl -out ca.crl -crl days 1
```

証明書に CDP 設定

```
crlDistributionPoints = URI:http://crl.example.com/ca.crl
```

課題:

リアルタイム性の欠如として、CRL 更新頻度 (例: 日次) により、失効から反映まで遅延が発生する。スケーラビリティとして、失効証明書が増えると CRL サイズが肥大化する (10,000 枚で数 MB)。ネットワーク負荷として、すべてのクライアントが CRL をダウンロードする必要がある。

(3) OCSP (Online Certificate Status Protocol) 方式

OCSP は、リアルタイムで証明書の状態を問い合わせるプロトコルである。

仕組み: クライアントは OCSP レスポンダに証明書のシリアル番号を送信し、レスポンダが現在の状態 (Good/Revoked/Unknown) を返答する。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

設定例: bash# OCSP レスポンダ起動

```
openssl ocsp -port 8080 ¥  
-index index.txt ¥  
-CA ca-cert.pem ¥  
-rsigner ocsp-cert.pem
```

証明書に OCSP 設定

```
authorityInfoAccess = OCSP;URI:http://ocsp.example.com:8080
```

利点: リアルタイム性が高く、スケーラビリティに優れ、ネットワーク効率が良い。

(4) OCSP Stapling

OCSP Stapling は、サーバが OCSP レスポンスを事前を取得し、TLS handshake 時にクライアントに送信する仕組みである。

メリット: クライアントのレイテンシ削減 (OCSP サーバへの往復通信が不要)、プライバシー保護 (クライアントのアクセス先が OCSP サーバに知られない)、OCSP サーバの負荷軽減が実現できる。

Apache 設定例: `apacheSSLUseStapling on`

`SSLStaplingCache "shmcb:/var/run/ocsp(128000)"`

(5) 実装推奨

小規模環境 (～100 証明書) では CRL のみで十分である。中規模環境 (100～1000 証明書) では CRL + OCSP の併用が推奨される。大規模環境 (1000 証明書以上) では OCSP + OCSP Stapling が最適である。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

- 結論

本演習「PKI に基づくデータ暗号化基盤の構築とセキュリティ評価」は、理論と実践を結びつける貴重な機会であった。設問 1 では、PKI の構成要素とデータ暗号化の関係を理論的に理解した。設問 2 では、実際に Private CA を構築し、Web サーバ証明書を発行することで、理論を実装に落とし込む経験を得た。設問 3 では、ブラウザでの証明書警告という現象を通じて、PKI の信頼モデルの本質を理解した。設問 4 では、CA 秘密鍵漏洩という最悪のシナリオを想定し、多層防御の重要性を認識した。設問 5 では、実運用を見据えた考察を通じて、技術選択の判断基準を習得した。これら 5 つの設問を通じて、PKI 技術は単なる「証明書を発行する仕組み」ではなく、現代のデジタル社会を支える信頼の基盤であることを深く理解できた。インターネット上のあらゆる安全な通信、E コマース、オンラインバンキング、リモートワーク、IoT、クラウドサービスのすべてが、PKI 技術に依存している。私たちが日常的に利用している「https://」で始まる Web サイトの背後には、世界中の CA が発行した数億枚の証明書と、それを支える膨大なインフラが存在する。技術は常に進化し、新たな脅威も出現するが、PKI の基本原理と「信頼」という概念は変わらない。この基盤の上に、継続的な学習と実践を積み重ねることで、より高度なセキュリティ専門家として成長していきたい。最後に、本演習を通じて、セキュリティ技術の奥深さと面白さを実感できたことに感謝したい。実際に手を動かして構築し、失敗し、試行錯誤する過程で、技術への理解が深まり、実務への応用力が身についた。

セキュリティ実践演習第一(2025 年度後期集中)
「データの暗号化」レポート課題

参考文献

- [1] RFC 5280, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile", <https://datatracker.ietf.org/doc/html/rfc5280>
- [2] RFC 8446, "The Transport Layer Security (TLS) Protocol Version 1.3", <https://datatracker.ietf.org/doc/html/rfc8446>
- [3] RFC 8555, "Automatic Certificate Management Environment (ACME)", <https://datatracker.ietf.org/doc/html/rfc8555>
- [4] RFC 6960, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP", <https://datatracker.ietf.org/doc/html/rfc6960>
- [5] RFC 6962, "Certificate Transparency", <https://datatracker.ietf.org/doc/html/rfc6962>
- [6] CA/Browser Forum, "Baseline Requirements for the Issuance and Management of Publicly-Trusted Certificates", <https://cabforum.org/baseline-requirements-documents/>
- [7] NIST, "Post-Quantum Cryptography Standardization", <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [8] NIST SP 800-57 Part 1 Rev. 5, "Recommendation for Key Management: Part 1 - General", <https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final>
- [9] OpenSSL Documentation, <https://www.openssl.org/docs/>
- [10] Apache HTTP Server mod_ssl Documentation, https://httpd.apache.org/docs/current/mod/mod_ssl.html
- [11] Let's Encrypt Documentation, <https://letsencrypt.org/docs/>
- [12] 独立行政法人情報処理推進機構（IPA）, 電子政府における調達のために参照すべき暗号のリスト（CRYPTREC 暗号リスト）, <https://www.cryptrec.go.jp/list.html>